NORTHSTAR HEALTH SERVICES

Healthcare Claims Data Platform

Technical Design Document

Prepared for: Northstar Health Services

Prepared by: Data Engineering & Analytics Practice

Document Classification: Confidential

Narrative Edition — technical design theory and rationale, presented without embedded diagrams or screenshots

Document ID: NHS-DATA-TDD-001 | Version 1.0 | Status: Technical Baseline

Related Architecture Reference: NHS-DATA-ARCH-001

Document Control

Revision History

Version	Date	Author	Status	Summary of Changes
1.0	Current	Data Engineering & Analytics Practice	Baseline	Initial technical design baseline established for the healthcare claims data platform, translating the approved solution architecture into implementable pipeline, storage, transformation, data-model, and reporting specifications.

Distribution and Confidentiality

This document is classified as Confidential and is intended solely for the internal use of Northstar Health Services and its authorized technical delivery partners. It contains sensitive architectural and operational detail regarding the handling of protected healthcare claims data and must not be reproduced or distributed outside the agreed project governance structure without prior written consent.

Purpose and Scope

This Technical Design Document (TDD) sets out the detailed engineering implementation of the Northstar Health Services claims data platform. It builds on the approved solution architecture (NHS-DATA-ARCH-001) and translates architectural intent into concrete pipeline design, storage layer specification, transformation logic, dimensional modeling, and reporting delivery. The document is intended to serve as the authoritative reference for build, test, deployment, and ongoing operational support of the platform, and as the basis for technical traceability back to architecture requirements.

The scope of this document covers the end-to-end data journey: from ingestion of daily claims extracts out of the Adroit operational claims system, through staged transformation across Bronze, Silver, and Gold layers within Microsoft Fabric, to the governed semantic layer and Power BI reporting experience used by revenue-cycle and finance stakeholders.

Table of Contents

1. Technical Overview

The Northstar Health Services claims data platform has been implemented as a staged, dependency-driven processing chain built on Microsoft Fabric. The design follows a medallion (Bronze/Silver/Gold) engineering pattern, ensuring that each layer of the platform has a single, well-defined responsibility and that data quality is progressively enforced as claims data moves from raw ingestion through to curated, analytics-ready structures.

Daily claims extracts are landed in Azure Blob Storage, ingested into an Azure SQL staging boundary, promoted through Bronze and Silver processing layers, and ultimately published into Gold analytical structures that are consumed by a governed semantic model and surfaced through Power BI. Orchestration, monitoring, and notification are managed centrally through a master pipeline, giving operations teams a single point of control and visibility over the daily processing cycle.

1.1 End-to-End Processing Flow

The technical processing chain implemented for the platform is summarized below:

Processing Chain

Adroit Operational Claims System → Daily Claims Extract → Azure Blob Storage → PL_Blob_To_SQL → Azure SQL Staging → PL_SQL_To_Bronze → Bronze Layer → PL_Bronze_To_Silver → Silver Layer → PL_Silver_To_Gold → Gold Warehouse → Fact and Dimension Tables → Semantic Model → Power BI

This layered approach was selected deliberately over a direct source-to-warehouse pattern. It decouples file availability from downstream processing, preserves a source-aligned copy of the data for audit and traceability, and isolates business-rule transformation from both raw ingestion and analytical serving. The result is a platform that is easier to troubleshoot, easier to extend, and better aligned with data governance expectations for healthcare claims information.

1.2 Design Principles

- Separation of concerns across ingestion, conformance, and serving layers.
- Dependency-driven orchestration so that no downstream stage executes against incomplete or unvalidated upstream data.
- Idempotent, business-key-driven processing to guarantee consistent outcomes on re-run.
- Centralized monitoring and notification to support a proactive, rather than reactive, operating model.
- Governed semantic abstraction between curated warehouse data and business-facing reporting.

2. Workspace and Component Inventory

All platform components are deployed within a dedicated Microsoft Fabric workspace, which acts as the deployment and operational boundary for the solution. The table below summarizes each component and its role within the platform.

Component	Technical Role
Microsoft Fabric Workspace	Deployment and operational boundary for all platform assets
PL_Master_Claims_Automation	Top-level orchestration pipeline
PL_Blob_To_SQL	Landing ingestion into Azure SQL staging
PL_SQL_To_Bronze	Promotion of staged data into the Bronze layer
PL_Bronze_To_Silver	Bronze transformation and Silver layer publication
PL_Silver_To_Gold	Silver-to-Gold analytical publication
Notebooks	Transformation and data engineering execution
Azure SQL Staging	Relational ingestion boundary
Bronze / Silver Storage (Lakehouse)	Layered engineering persistence
Gold Warehouse	Curated analytical serving layer
Semantic Model	Governed analytical relationships and measures
Power BI	Business reporting and stakeholder consumption
Power Automate	Operational notifications

Fabric workspace inventory showing lakehouse, notebook, and report assets deployed for the RCM-Analytics-DEV workspace.

Fabric workspace component listing, confirming pipeline, semantic model, and warehouse artifacts.

3. Pipeline Architecture

Execution across the platform is controlled by a master orchestration pipeline that enforces activity-level dependencies. Each downstream processing layer is gated on the successful completion of its upstream stage, ensuring that partial or failed loads cannot silently propagate into curated reporting structures.

Master orchestration pipeline (PL_Master_Claims_Automation) showing the sequenced dependency chain across ingestion, transformation, and notification activities.

The responsibilities of each pipeline within the chain are summarized below.

Pipeline	Input	Output	Technical Responsibility
PL_Blob_To_SQL	Azure Blob Storage	Azure SQL Staging	Ingests the daily claims file into the relational staging boundary
PL_SQL_To_Bronze	Azure SQL Staging	Bronze Layer	Persists source-aligned data for traceability

Pipeline	Input	Output	Technical Responsibility
PL_Bronze_To_Silver	Bronze Layer	Silver Layer	Applies transformation, cleansing, and standardization
PL_Silver_To_Gold	Silver Layer	Gold Warehouse	Publishes curated analytical structures

4. Pipeline-Level Design

4.1 PL_Blob_To_SQL

This pipeline establishes the ingestion boundary between the daily file landing zone and the relational staging environment. It is the first point at which raw claims data enters the managed platform and is therefore subject to explicit contract and pre-load controls.

Design Attribute	Implementation
Source	Azure Blob Storage
Input	Daily claims data file
Activity Pattern	Copy Data activity
Destination	Azure SQL staging table
Pre-load Control	Staging preparation / truncate procedure, where configured
Data Contract	Source-to-target column mapping
Failure Outcome	Pipeline activity failure with operational notification

Copy Data activity configuration for PL_Blob_To_SQL, showing the Azure SQL destination and staging table target.

Source-to-target column mapping applied during the Blob-to-SQL ingestion step.

Staging preparation (truncate) stored-procedure step, executed ahead of each daily load to guarantee a clean staging boundary.

4.2 PL_SQL_To_Bronze

This pipeline promotes staged relational data into the Bronze layer, establishing the first durable, source-aligned representation of the data within the Fabric lakehouse.

Design Attribute	Implementation
Source	Azure SQL staging
Target	Bronze layer (Lakehouse)
Pattern	Copy activity with controlled promotion

Design Attribute	Implementation
Purpose	Create a source-aligned, downstream-persistent copy of the data

Source configuration for PL_SQL_To_Bronze, reading from the Azure SQL staging table.

Bronze layer destination configuration within the LH_RCM_Bronze lakehouse.

Column-level mapping and type conversion settings applied during SQL-to-Bronze promotion.

4.3 PL_Bronze_To_Silver

This pipeline executes the conformance and transformation stage of the platform. The Silver layer represents the point at which source-aligned data is converted into standardized, analytics-ready data through notebook-based transformation logic. Key responsibilities at this stage include cleansing, standardization, type handling, deduplication, and the application of business rules.

Design Attribute	Implementation
Source	Bronze layer
Target	Silver layer
Processing Engine	Fabric notebook (PySpark) transformation logic
Key Responsibilities	Cleansing, standardization, type handling, and business-rule application

Bronze-to-Silver notebook logic performing deduplication on the composite claim-line key prior to an upsert into the Silver Delta table.

4.4 PL_Silver_To_Gold

This pipeline publishes curated analytical data from the Silver layer into the Gold warehouse serving layer, producing the fact and dimension structures consumed by the semantic model.

Design Attribute	Implementation
Source	Silver layer
Target	Gold Warehouse
Processing	Curated analytical publication
Output	Fact and dimension structures

Gold warehouse destination configuration, targeting the staging claims table ahead of upsert into curated Gold structures.

Silver-to-Gold column mapping and type conversion configuration.

5. Activity-Level Design

5.1 Master Orchestration Activities

The master pipeline sequences a consistent set of activity patterns each time it executes, giving the platform a predictable and auditable daily operating rhythm.

Activity Pattern	Technical Function
Staging Preparation	Prepares the target staging boundary ahead of the daily load
Copy Claims	Moves daily source data into Azure SQL staging
SQL-to-Bronze	Promotes source-aligned records into the Bronze layer
Bronze-to-Silver	Executes transformation and standardization logic
Silver-to-Gold	Publishes curated analytical data to the Gold warehouse
Notification	Communicates the execution outcome to operations stakeholders

Activity-level run history for PL_Master_Claims_Automation, confirming sequential execution and successful completion of each stage.

Confirmed successful end-to-end pipeline execution across all six orchestrated activities.

6. Azure Blob Ingestion Design

Azure Blob Storage serves as the file-landing interface for daily claims data and is treated as the authoritative source boundary for the ingestion pipeline. Landing the file independently of the processing pipeline decouples file availability from pipeline execution, which materially simplifies recovery in the event of a late or missing source extract.

Raw claims container within the Fabric-linked storage account, used as the landing zone for daily extracts.

Daily claims files landed in the raw-claims container, one blob per operating day.

6.1 Technical Controls

- File availability must be confirmed before ingestion is triggered.
- Source columns must conform to the agreed mapping contract.
- Date and numeric values must be compatible with the target data types before conversion.
- A failed ingestion must not be permitted to propagate to downstream, uncontrolled publication.

7. Azure SQL Staging Design

Azure SQL staging provides a controlled relational boundary between raw file ingestion and Bronze layer processing. It exists purely as a transient landing structure; no analytical consumption occurs directly against this layer.

Area	Technical Design
Purpose	Temporary staging persistence for source-aligned claims data
Load Pattern	Daily ingestion via the orchestration pipeline
Preparation	Staging reset / truncate activity, where configured
Validation	Source-to-target mapping and data type compatibility checks
Downstream Consumer	PL_SQL_To_Bronze

Design Control: Date Conversion

Source date values such as '29-06-2026' must be standardized or explicitly parsed prior to conversion into the target Date/DateTime type. This is treated as a source-to-target data contract control rather than a downstream reporting concern, and is enforced at the earliest practical point in the pipeline.

8. Bronze Layer Design

The Bronze layer preserves a source-aligned representation of ingested claims data, providing an auditable foundation for all downstream transformation and enabling reprocessing without returning to the original source system.

Characteristic	Design
Granularity	Claims source record / claim-line representation
Purpose	Traceability and source-aligned persistence
Transformations Applied	Minimal structural transformation only
Downstream Consumer	Silver transformation process
Control	Record-count reconciliation and load validation

9. Silver Layer Design

The Silver layer is the conformed data engineering layer of the platform. It is the point at which raw, source-aligned data is transformed into clean, standardized, business-rule-conformant data suitable for analytical publication.

Transformation Area	Technical Rule
Date Standardization	Convert source date representations to target-compatible date values
Numeric Standardization	Validate and normalize decimal and numeric fields
Null Handling	Apply defined handling logic for required attributes
Deduplication	Validate logical claim-line uniqueness
Business Rules	Apply conformance rules ahead of Gold publication
Incremental Processing	Use the logical business key to drive insert / update behavior

10. Gold Layer Design

The Gold layer is the curated analytical serving layer of the platform. It exposes fact and dimension structures, modeled in a star schema, to the governed semantic model that underpins all Power BI reporting.

Object Type	Technical Purpose
Fact Table	Stores measurable claim-line activity
Dimension Tables	Provide descriptive analytical context
Surrogate Keys	Support stable, warehouse-internal relationships
Business Keys	Maintain source system identity and matching
Warehouse Layer	Provides reporting-ready analytical structures

Gold layer star schema, showing Fact_Claims related to Dim_Facility, Dim_Payer, and Dim_Provider through surrogate-key relationships.

11. Incremental Processing and Business Keys

The platform identifies a logical claim line using a composite business key consisting of Claim_ID and Line_Number. This key underpins all incremental processing logic across the Silver and Gold layers and ensures that repeated processing of the same source data produces a stable, idempotent outcome.

Record State	Technical Action
New business key	INSERT
Existing business key with changed attributes	UPDATE
Repeated business key with no change	No additional logical claim line is created

Design Boundary

The business key governs claim-line identity across source and conformed layers. Dimension surrogate keys are separate, warehouse-internal keys and must not be confused with source business identifiers; conflating the two is a common source of downstream relationship errors and is

Record State	Technical Action
explicitly guarded against in the Gold model design.	

12. Data Quality and Validation

Data quality is enforced as both a technical control embedded within the pipeline and as an operational release gate before reporting data is considered current and fit for business consumption.

Validation	Expected Control
Business Key Completeness	Claim_ID and Line_Number must be present on every record
Duplicate Check	No uncontrolled duplicates on the composite business key
Date Validation	Source date values conform to target parsing rules
Numeric Validation	Amounts and quantities convert successfully to target types
Row Count Reconciliation	Record counts reconcile across major processing stages
Referential Integrity	Fact table foreign keys resolve correctly to dimension tables
Pipeline Status	Downstream execution only proceeds on successful upstream completion

13. Stored Procedure and Transformation Design

Stored procedures and notebook-based transformations are treated as first-class implementation components of the platform and are documented consistently by purpose, inputs, sources, targets, business rules, and failure behavior. This ensures that transformation logic remains transparent, testable, and independently maintainable from orchestration and reporting layers.

Component	Required Technical Documentation
Stored Procedure	Name, parameters, source tables, target tables, transformation logic, transaction and error behavior
Notebook	Purpose, inputs, transformation logic, outputs, dependencies, runtime parameters
Pipeline Activity	Activity type, source, sink, dependency chain, retry and failure behavior

This layered documentation approach keeps transformation logic cleanly separated from orchestration control and downstream reporting consumption, which supports independent testing and reduces the blast radius of any single component change.

14. Dimensional Model Technical Design

The Gold layer implements a star-schema dimensional model centered on a single claims fact table, surrounded by descriptive dimension tables. This pattern was selected to support flexible, performant analytical querying and to give the semantic model a simple, well-understood set of relationships to govern.

Dimensional model view within the Fabric semantic model, showing Fact_Claims related to Dim_Date, Dim_Facility, Dim_Patient, Dim_Payer, and Dim_Provider.

Model Component	Technical Role
Fact_Claims	Claim-line measures and dimension foreign keys
Dim_Payer	Payer descriptive attributes
Dim_Member	Member (patient) descriptive attributes
Dim_Provider	Provider descriptive attributes
Dim_Date	Date analysis and time-based slicing

Fact-to-dimension relationships are governed exclusively through warehouse surrogate keys, while source business identifiers are retained on each table for traceability and downstream matching purposes.

15. Semantic Model and Power BI Design

The semantic model provides the governed analytical abstraction layer between the curated Gold warehouse and Power BI reporting. Centralizing relationships and measures within the semantic model ensures that business logic is defined once and consumed consistently across every downstream report.

Layer	Technical Responsibility
Gold Warehouse	Curated facts and dimensions
Semantic Model	Relationships, measures, and analytical logic
Power BI	Visual consumption and stakeholder reporting

Executive Dashboard, providing a consolidated view of total claims, charges, allowed amount, paid amount, balance, and denial and rejection rates.

Operational Performance Dashboard, tracking claims processing volumes, productivity, and processing trend over time.

Payer Performance Dashboard, presenting charges, paid amounts, collection rate, and denial rate broken down by payer.

Financial Performance Dashboard, presenting revenue, collections, outstanding balance, and revenue flow analysis.

Denials & Rejections Dashboard, surfacing denial rate trend, top denial reasons, and denial and rejection breakdowns by facility, payer, and provider.

16. Scheduling, Monitoring and Notifications

The platform operates on a scheduled daily execution cycle. Pipeline status, recent run history, and successful completion are actively monitored as part of the standard daily operating rhythm, with Power Automate used to route operational notifications to the support team.

Scheduling and failure-notification configuration for PL_Master_Claims_Automation, including the daily recurrence and notification recipient settings.

Fixed daily schedule configuration, confirming recurrence, start/end dates, and time zone settings.

Recent pipeline run history confirming successful completion of the daily orchestration.

RCM Alert Center report, combining Power BI and Power Automate to surface daily claims summary metrics and trigger operational alerts.

17. Error Handling and Recovery

Failure handling across the platform follows a consistent, controlled troubleshooting pattern designed to minimize mean time to resolution while preserving data integrity.

- Identify the failed pipeline and the specific failing activity.
- Capture the error code and the failing data element.
- Determine whether the root cause lies in source data, mapping configuration, data type handling, connectivity, or transformation logic.
- Correct the root cause through a controlled change process.
- Rerun the affected processing stage.
- Validate downstream data completeness and reporting freshness before closing the incident.

Known Failure Pattern: Date Conversion

A date conversion failure can occur when the source date string format is not recognized by the target DateTime conversion routine. The corrective control is to standardize or explicitly parse the source value prior to target conversion, rather than relying on implicit type coercion.

18. Security and Access Design

Security design for the platform follows the principle of least privilege and is aligned to the sensitivity of the healthcare claims data being processed.

- Role-based access control is applied to all Fabric workspaces and reporting assets.
- SQL and storage access is restricted to the required service identities and named users only.
- Credentials are never embedded in notebooks, pipeline definitions, or documentation.
- Connection secrets are protected through approved secret-management mechanisms.

- Data access controls appropriate to healthcare claims information are applied consistently across all layers.

19. Deployment Design

Technical components are promoted through a controlled, multi-environment release process to ensure changes are validated before reaching production.

Deployment Stage	Activities
Development	Build and unit-level validation
QA / Test	Integration testing, data-quality validation, and regression testing
Production	Controlled release, scheduling activation, and operational validation

Deployment order is designed to preserve upstream dependencies: database objects and storage structures are deployed first, followed by pipelines and notebooks, with the semantic model and reporting assets deployed last.

20. Technical Risks and Controls

Risk	Control
Schema drift	Source contract and mapping validation
Date format mismatch	Explicit parsing and type validation
Duplicate claim lines	Composite business-key validation
Dimension key mismatch	Controlled dimension loading and relationship validation
Partial pipeline completion	Dependency-driven orchestration
Reporting freshness issue	Run monitoring and automated notifications

21. Technical Decisions

The following key technical decisions were made during the design of the platform, each reflecting a deliberate trade-off assessment against alternative approaches.

Decision	Technical Rationale
Use Blob Storage as the landing interface	Decouples file availability from pipeline processing
Use Azure SQL for staging	Provides a controlled relational ingestion boundary
Use layered Bronze / Silver / Gold processing	Separates preservation, conformance, and serving responsibilities

Decision	Technical Rationale
Use Claim_ID + Line_Number as the business key	Supports reliable claim-line-level matching and idempotent processing
Use a dimensional Gold model	Supports flexible analytical reporting and reusable semantic relationships
Use a semantic model ahead of Power BI	Centralizes analytical logic for consistent reuse across reports
Use scheduled orchestration	Supports predictable, repeatable daily operations

Appendix A. Technical Traceability

The table below maps each architecture requirement to its corresponding implementation evidence within this technical design, supporting audit and governance review of the delivered solution.

Architecture Requirement	TDD Implementation Evidence
Daily ingestion	Blob landing zone and PL_Blob_To_SQL
Relational staging	Azure SQL staging
Bronze processing	PL_SQL_To_Bronze
Silver transformation	PL_Bronze_To_Silver
Gold publication	PL_Silver_To_Gold
Incremental processing	Claim_ID + Line_Number composite business key
Analytical model	Fact and dimension structures (star schema)
Reporting	Semantic model and Power BI
Operations	Scheduling, run monitoring, and notifications

End of Document